

Edge AI PPA Estimator: A Hardware-Aware Dashboard for Model Deployment

Submitted by:

Niket Agrawal (M23CSA520)

Syam Krishnan Shathidharan (M23CSA535)

1. Introduction & Problem Statement

Deploying machine learning models to edge devices—whether it is a Raspberry Pi or a custom Nvidia Jetson—is rarely a straightforward process. Developers usually train their models on powerful cloud servers with plenty of memory. But when it comes time to move those weights onto physical, resource-constrained silicon, things tend to break. Engineers frequently run into Out-of-Memory (OOM) errors, unexpected latency spikes, or thermal throttling.

The core issue is that edge hardware is highly heterogeneous. Traditional machine learning algorithms (like Random Forests) can easily bottleneck a CPU by thrashing its cache due to unpredictable memory access patterns. On the other hand, Deep Learning models usually hit a wall at the memory bus long before they max out the actual compute cores. Trying to fix these issues through trial and error—compiling, flashing to the device, crashing, and starting over—wastes a massive amount of engineering time.

What MLOps engineers really need is a way to accurately estimate hardware constraints *before* they ever touch the actual device. That is exactly what the Edge AI Deployment IDE is designed to do. It acts as an upfront simulation dashboard that predicts Power, Performance, and Area (PPA). By modeling physical cache penalties, simulating post-training quantization, and generating visual Roofline models, this tool allows developers to validate their architectures and generate deployment scorecards in seconds instead of days.

2. System Architecture & Methodology

Real-world edge devices are heterogeneous, containing both a general-purpose CPU and a dedicated AI Accelerator (NPU/GPU). To accurately simulate this, the IDE is divided into two distinct routing branches governed by specific mathematical methodologies.

2.1. Classical ML Branch (Edge CPU Target)

This branch evaluates traditional machine learning models (Decision Trees, Random Forests, SVMs) which execute sequentially on the CPU.

- **Resource Estimation:** Estimates inference operations and Static Random-Access Memory (SRAM) usage.

- **Complexity Analysis:** Incorporates dynamic Big O calculations to illustrate memory growth, particularly for tree-based models where memory usage can increase exponentially with depth (2^{depth}).
- **Dynamic Memory Locality & Cache Modeling:** The engine compares the model's footprint against target L1/L2 cache limits. It applies a 4-cycle penalty for L1 hits, a 12-cycle penalty for L2 hits, and a severe 200-cycle penalty if the model exceeds cache bounds and thrashes into main DRAM, accurately simulating real-world CPU pipeline stalls.

2.2. Deep Learning Branch (Edge NPU/GPU Target)

This branch evaluates dense matrix multiplications characteristic of Convolutional Neural Networks (CNNs).

- **Model Profiling:** Utilizes PyTorch's `thop` library to trace network graphs and extract operational metrics such as Multiply-Accumulate Operations (MACs) and Floating Point Operations (FLOPs). `torchinfo` is used for detailed layer-by-layer architectural inspection.
- **Roofline Model Application:** A core component that assesses whether a Convolutional Neural Network (CNN) is memory-bound or compute-bound. This is achieved by comparing the model's Arithmetic Intensity (FLOPs/Bytes) against the hardware's Ridge Point (Peak Compute/Memory Bandwidth).
- **Resource Estimation:** Calculates total model parameters, estimated memory footprint, and silicon area.
- **Performance Metrics:** Predicts latency, throughput (Inference/Sec), and power draw based on hardware capabilities and the Roofline model's insights.
- **ONNX Support:** Provides parsing capabilities for ONNX models to extract operation types and parameter counts, enabling analysis for pre-converted models. For custom pre-converted models, the system utilizes `onnx.shape_inference` to dynamically walk the computational graph, extract exact tensor shapes, and mathematically calculate MAC operations based on spatial dimensions ($H_{\text{out}} * W_{\text{out}} * C_{\text{out}} * C_{\text{in}} * K_h * K_w$ for Convolutions).

2.3. Automated PPA Deliverables

Finally, to make this a complete end-to-end MLOps tool, we integrated an in-memory PDF generator using `reportlab`. Once a simulation is complete, the IDE packages the physical footprints, execution metrics, and any hardware violation alerts into a clean, professional deployment scorecard that can be immediately handed off to the hardware engineering team.

3. Technical Implementation & User Interface

To bridge the theoretical mathematical models with an intuitive developer experience, the IDE is structured around a robust Python backend and an interactive frontend.

3.1. Technical Stack

The system architecture relies on the following libraries and frameworks:

- **Frontend:** `Streamlit` is utilized to rapidly deploy the interactive, web-based PPA dashboard.
- **Backend & Profiling (Python):** `PyTorch` handles native Deep Learning model definitions and tensor operations.
 - `thop` and `torchinfo` are integrated for dynamic model profiling (extracting MACs/FLOPs) and granular, layer-by-layer architectural inspection.
 - `onnx` and `onnx.shape_inference` provide the protobuf parsing engine, allowing the system to securely intercept and mathematically analyze pre-converted graph definitions in memory.
- **Deployment Integration:** `pyngrok` is configured to expose the local Streamlit server via a secure public URL, enabling seamless execution and testing directly from a Google Colab environment.
- **Data & Visualization:** `pandas` manages the structuring of spatial dimensions and operator counts, while `matplotlib` dynamically renders the visual Roofline performance curves.
- **Automated Reporting:** `reportlab` powers the in-memory PDF generation engine, dynamically drawing the deployment scorecards without writing temporary files to the server.

3.2. User Workflow & Constraint Inputs

- The user interface guides the developer through a precise, hardware-aware configuration process before executing the simulation:
- **Hardware Capabilities Input:** The user defines the physical realities of the target Edge silicon. This includes Peak Compute (TOPS/GFLOPS), Memory Bandwidth (GB/s), Clock Speed (MHz), Device Base Power (Watts), and the specific **L1 and L2 Cache Sizes (KB)** of the CPU.
- **Project Constraints Input (PPA):** Strict deployment guardrails are established by specifying maximum allowable limits for Memory (MB), Latency (ms), Total Power (Watts), and Silicon Area (mm²).
- **Precision & Quantization Setup:** Before running the simulation, the user selects the target precision level (e.g., FP32 Baseline, FP16, INT8, or INT4). This allows the engine to simulate post-training quantization dynamically.

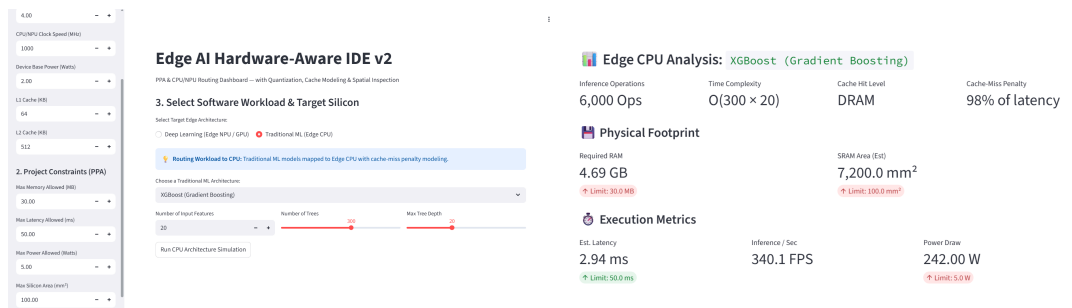
- **Workload & Model Selection:** The user toggles the master routing architecture, choosing between "Deep Learning (Edge NPU / GPU)" or "Traditional ML (Edge CPU)".
 - *Traditional ML:* The developer selects the specific algorithm (e.g., Decision Tree, Random Forest) and tunes the architectural hyperparameters (e.g., number of input features, max tree depth, number of estimators).
 - *Deep Learning:* The developer selects a baseline edge vision model (e.g., MobileNetV2, ResNet18) or securely uploads a custom `.pth` or `.onnx` file.
- **Simulation, Analysis & Export:** Upon execution, the IDE renders comprehensive metrics (workload profiling, physical footprint, and execution latency). It generates the visual Roofline plot and triggers the "Hardware-Aware Optimization Engine." This engine provides actionable advice—such as explicitly recommending INT8 quantization for memory-bound models or warning of DRAM cache-thrashing for oversized Random Forests. Finally, the user can instantly download these insights as a formalized PDF scorecard.

4. Implementation Results

The following case studies demonstrate the theoretical methodology being applied in real-time, actively catching hardware violations and diagnosing bottlenecks before physical deployment.

4.1. Traditional ML Heuristics and Exponential Memory Traps

4.1.1. XGBOOST



To validate the Edge CPU routing branch, an XGBoost (Gradient Boosting) model was configured to simulate a highly unoptimized architecture. The model was defined with an ensemble of 300 trees, each with a maximum depth of 20. The target hardware was constrained to a strict 30.0 MB memory limit, a 5.0 W power ceiling, and an area of 100.0 mm². To test the new memory locality engine, the CPU was given a standard 64 KB L1 Cache and a 512 KB L2 Cache.

There is a common misconception in MLOps that traditional ML models are inherently "lighter" than deep neural networks. The IDE immediately disproves this. It calculated the exact time complexity bound as $O(300 * 20)$, yielding 6,000 inference operations.

More critically, the IDE evaluated the physical footprint. Because decision trees rely on conditional branching, their memory requirements grow exponentially (2^{depth}). The engine accurately calculated that this specific architecture would require a massive 4.69 GB of RAM (and 7,200.0 mm² of SRAM area)—an impossible figure for edge silicon.

Because this 4.69 GB footprint vastly exceeds the 512 KB L2 Cache, the IDE's dynamic memory locality engine detected severe "Cache Thrashing." It correctly diagnosed the cache hit level as Main Memory (DRAM), calculating that a staggering 98% of the model's latency would be wasted purely on memory fetch penalties rather than actual compute.

 CPU Hardware-Aware Optimization

🔥 **Memory Overflow:** The tree depth or ensemble size requires 4800.0 MB, exceeding the allocated 30.0 MB Edge RAM.

💡 **Action:** Restrict `max_depth` (pruning) or reduce `n_estimators`. Memory for tree algorithms grows exponentially (2^{depth}).

🔥 **Power Failure:** Power consumption exceeds the physical budget of the device.

💡 **Action:** Reduce model size or apply dynamic voltage and frequency scaling (DVFS) to the Edge CPU.

🔥 **Constraints Violated:** Failed 2 hardware checks.

📄 Export PPA Scorecard

📄 Download PPA Scorecard (PDF)

Edge AI PPA Scorecard

Generated: 2026-04-18 10:00:57

Hardware: 1.0 TOPS • 4.0 GB/s BW • 1000 MHz • 2.0 W base • L1 64 KB • L2 512 KB
Constraints: Memory ≤30.0 MB • Latency ≤50.0 ms • Power ≤5.0 W • Area ≤100.0 mm²

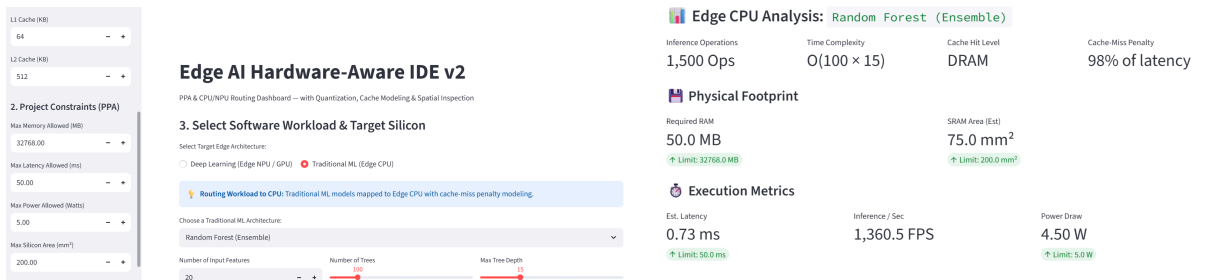
Metric	Value	Constraint Check
Model Type	XGBoost (Gradient Boosting)	INFO
Required RAM	4800.0 MB	FAIL
Latency	2.94 ms	PASS
Power Draw	242.00 W	FAIL

❌ CONSTRAINTS VIOLATED (2 checks failed)

Consequently, the Hardware-Aware Optimization Engine successfully intercepted these violations. It triggered critical "Memory Overflow" and "Power Failure" (242.00 W) alerts. Acting as an expert system, the IDE advised the developer to prune the `max_depth` to stop the exponential memory explosion and suggested applying dynamic voltage and frequency scaling (DVFS) to manage the power draw.

Finally, as shown in the exported PPA Scorecard, the system compiled these diagnostics into a formal, downloadable PDF deliverable, permanently logging the two failed hardware checks and blocking the unviable compilation.

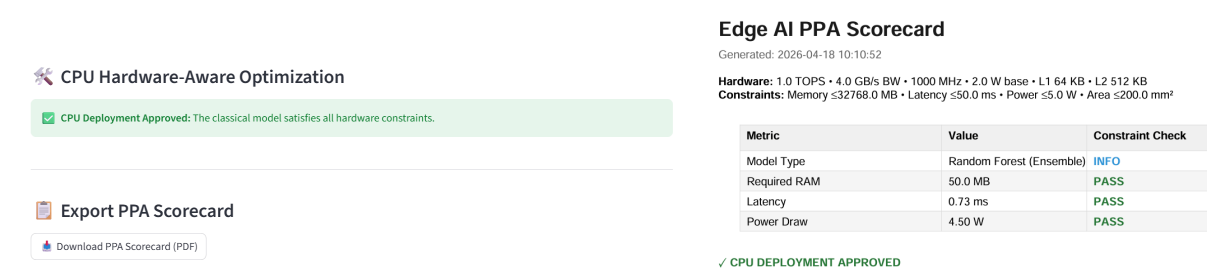
4.1.2. Random Forest



To contrast the previous failure, a second simulation was executed using a Random Forest ensemble configured with 100 trees and a maximum depth of 15. For this scenario, we simulated a much more capable edge gateway, relaxing the global memory limit to 32,768.00 MB (approx. 32 GB) while maintaining a strict 50.0 ms latency and 5.0 W power budget.

The IDE calculated the time complexity at $O(100 \times 15)$, resulting in a highly optimized 1,500 inference operations. The physical footprint was estimated at a highly manageable 50.0 MB of RAM.

What makes this simulation particularly interesting is the IDE's memory locality tracking. Even though 50.0 MB easily satisfies the massive 32 GB system limit, it still vastly exceeds the CPU's internal 512 KB L2 Cache. The engine dynamically caught this architectural reality, noting that the Cache Hit Level remained at the DRAM level, meaning 98% of the execution latency is inherently tied to cache-missing memory fetches rather than pure CPU operations.

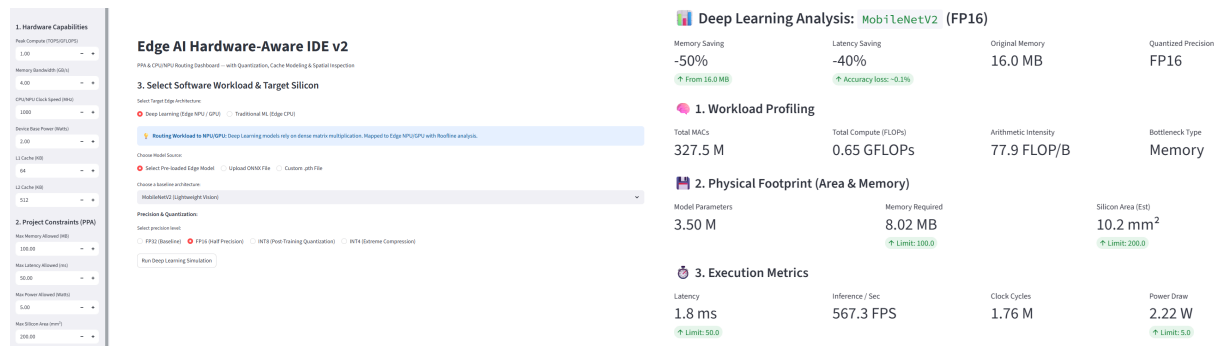


However, because the baseline operations were so low to begin with (1,500 Ops), the absolute estimated latency peaked at only 0.73 ms, maintaining a blistering throughput of 1,360.5 FPS. Since the latency and the 4.50 W power draw stayed well within our predefined physical guardrails, the Hardware-Aware Optimization Engine formally green-lit the architecture.

The exported PPA Scorecard reflects a perfect suite of "PASS" conditions. This case study demonstrates that the IDE does not blindly reject architectures purely for cache thrashing; rather, it contextualizes those architectural penalties against the actual, absolute physical limits of the edge device to approve viable deployments.

4.2. Deep Learning & Roofline Validation

4.2.1 MobileNetV2 : Memory Bottlenecks & FP16 Quantization



To evaluate the Deep Learning (NPU/GPU) routing branch, we selected MobileNetV2, a standard lightweight vision architecture. We set a relaxed memory limit of 100.0 MB and a latency ceiling of 50.0 ms.

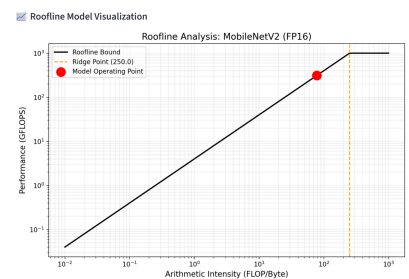
First, the IDE engaged its dynamic PyTorch profiler. Rather than using static estimations, it physically mapped the computational graph, extracting a computational load of 327.5 Million MACs (0.65 GFLOPs) and 3.50 Million parameters. At baseline (FP32), this model would require 16.0 MB of memory. To optimize the deployment, we toggled the IDE's simulation engine to FP16 (Half Precision). The system instantly recalculated the physics, demonstrating a 50% memory savings (dropping to 8.02 MB) and a 40% reduction in latency (down to an impressive 1.8 ms).

Hardware-Aware Optimization Engine

Diagnostic: Model is Memory-Bound. Arithmetic Intensity (77.9) is lower than the Hardware Ridge Point (250.0).

Action: Already on FP16. Consider Knowledge Distillation or Structural Pruning to further reduce payload size before it hits the memory bus.

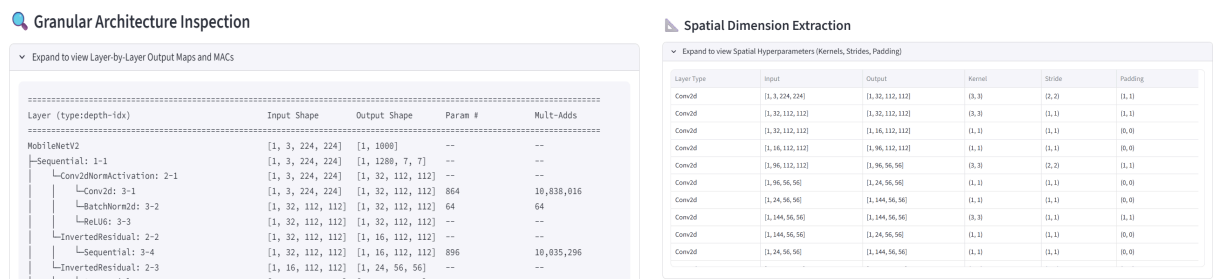
Deployment Approved: The architecture completely satisfies the target hardware PPA constraints.



The most critical insight, however, came from the IDE's visual Roofline Model. The engine calculated the model's Arithmetic Intensity at 77.9 FLOP/Byte. Because this metric is significantly lower than the simulated hardware's Ridge Point of 250.0, the IDE accurately plotted the model's operating point (the red dot) directly on the sloping memory bandwidth ceiling.

The Hardware-Aware Optimization Engine translated this graph into an explicit MLOps diagnostic: the model is strictly Memory-Bound. Even though the simulated NPU has the

compute capability (TOPS) to run the math faster, it is physically starved for data moving across the memory bus.



Edge AI PPA Scorecard

Generated: 2026-04-18 10:21:55

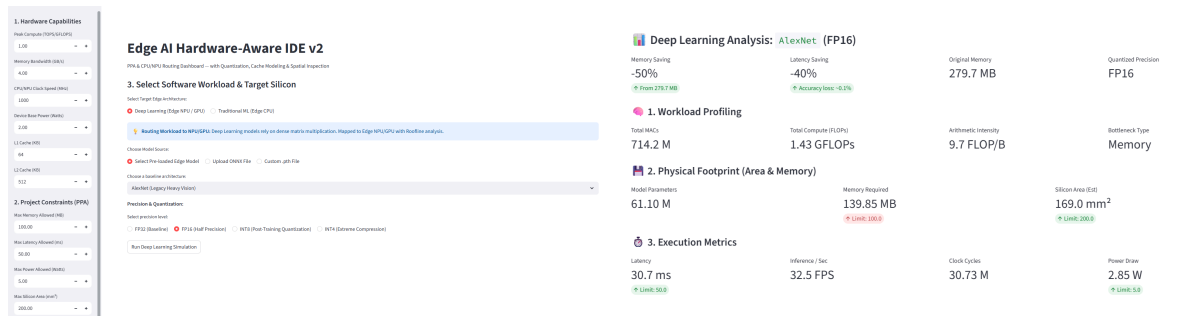
Hardware: 1.0 TOPS • 4.0 GB/s BW • 1000 MHz • 2.0 W base • L1 64 KB • L2 512 KB
Constraints: Memory ≤100.0 MB • Latency ≤50.0 ms • Power ≤5.0 W • Area ≤200.0 mm²

Metric	Value	Constraint Check
Model	MobileNetV2	INFO
Precision	FP16	INFO
Memory Required	8.0 MB	PASS
Silicon Area	10.2 mm²	PASS
Latency	1.8 ms	PASS
Power Draw	2.22 W	PASS

✓ DEPLOYMENT APPROVED

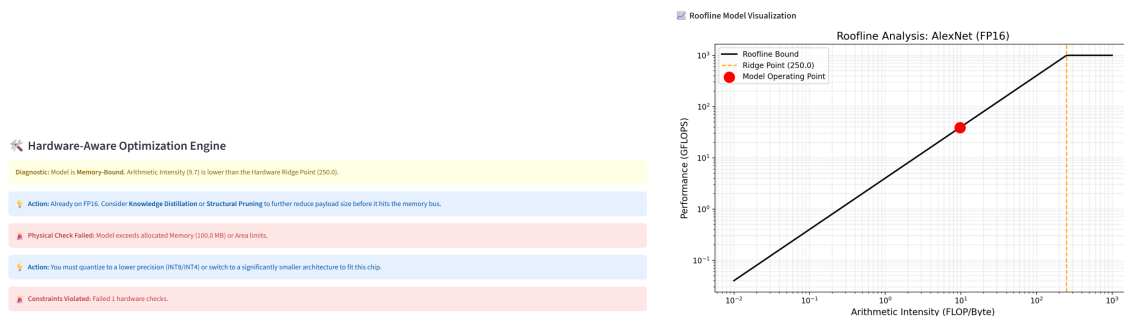
Because our FP16 quantization successfully compressed the payload enough to satisfy all physical constraints (yielding a massive throughput of 567.3 FPS at only 2.22 W), the engine officially approved the deployment. However, acting as a true advisory tool, the IDE noted that we were already using FP16 and actively recommended techniques like Knowledge Distillation or Structural Pruning to further optimize the memory bus bottleneck.

4.2.2 AlexNet Profiling and Bottlenecks



To push the IDE's constraint validation to its limits, we simulated a deployment using a legacy, parameter-heavy architecture: AlexNet. We maintained the same strict edge constraints as the previous NPU test (100.0 MB Memory, 50.0 ms Latency, 5.0 W Power). To give the model the best chance of fitting, we initialized the simulation with FP16 (Half Precision) quantization already applied.

The dynamic profiler immediately highlighted the architectural bloat of older CNNs. The IDE mapped a staggering 61.10 Million parameters and 1.43 GFLOPs of compute. While the FP16 quantization successfully slashed the baseline memory from a massive 279.7 MB down to 139.85 MB, and reduced the latency to a passing 30.7 ms, the sheer size of the model weights remained a critical issue.



The Roofline Model visualization tells the complete story of why this architecture fails on edge silicon. The IDE calculated an incredibly low Arithmetic Intensity of just 9.7 FLOP/Byte. Because the model has massive fully connected layers but relatively low computational reuse, the operating point (the red dot) is plotted far down the memory bandwidth slope. The system correctly diagnosed that the NPU's compute cores are completely starved, waiting on 139.85 MB of data to cross the bus.

Consequently, the Hardware-Aware Optimization Engine intercepted the deployment. It flagged a "Physical Check Failed," explicitly noting that the 139.85 MB footprint breached the hard 100.0 MB limit. As an advisory system, it informed the developer that since FP16 was insufficient, they

must either push for extreme INT8/INT4 quantization or abandon AlexNet entirely in favor of a modern, depthwise-separable architecture.

Granular Architecture Inspection

Expand to view Layer-by-Layer Output Maps and MACs

Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Mult-Adds
AlexNet	[1, 3, 224, 224]	[1, 1000]	--	--
↳Sequential: 1-1	[1, 3, 224, 224]	[1, 256, 6, 6]	--	--
↳Conv2d: 2-1	[1, 3, 224, 224]	[1, 64, 55, 55]	23,296	70,470,400
↳ReLU: 2-2	[1, 64, 55, 55]	[1, 64, 55, 55]	--	--
↳MaxPool2d: 2-3	[1, 64, 55, 55]	[1, 64, 27, 27]	--	--
↳Conv2d: 2-4	[1, 64, 27, 27]	[1, 192, 27, 27]	307,392	224,880,768
↳ReLU: 2-5	[1, 192, 27, 27]	[1, 192, 27, 27]	--	--
↳MaxPool2d: 2-6	[1, 192, 27, 27]	[1, 192, 13, 13]	--	--
↳Conv2d: 2-7	[1, 192, 13, 13]	[1, 384, 13, 13]	663,936	112,205,184
↳ReLU: 2-8	[1, 384, 13, 13]	[1, 384, 13, 13]	--	--
↳Conv2d: 2-9	[1, 384, 13, 13]	[1, 256, 13, 13]	884,992	149,563,648
↳ReLU: 2-10	[1, 256, 13, 13]	[1, 256, 13, 13]	--	--
↳Conv2d: 2-11	[1, 256, 13, 13]	[1, 256, 13, 13]	590,080	99,723,520

Spatial Dimension Extraction

Expand to view Spatial Hyperparameters (Kernels, Strides, Padding)

Layer Type	Input	Output	Kernel	Stride	Padding
Conv2d	[1, 3, 224, 224]	[1, 64, 55, 55]	[5, 5]	[4, 4]	[2, 2]
MaxPool2d	[1, 64, 55, 55]	[1, 64, 27, 27]	3	2	0
Conv2d	[1, 64, 27, 27]	[1, 192, 27, 27]	[5, 5]	[1, 1]	[2, 2]
MaxPool2d	[1, 192, 27, 27]	[1, 192, 13, 13]	3	2	0
Conv2d	[1, 192, 13, 13]	[1, 384, 13, 13]	[5, 5]	[1, 1]	[2, 2]
Conv2d	[1, 384, 13, 13]	[1, 256, 13, 13]	[5, 5]	[1, 1]	[2, 2]
Conv2d	[1, 256, 13, 13]	[1, 256, 13, 13]	[5, 5]	[1, 1]	[2, 2]
MaxPool2d	[1, 256, 13, 13]	[1, 256, 6, 6]	3	2	0

By expanding the Granular Architecture Inspection, we can visually verify the source of the bottleneck: massive 11x11 and 5x5 convolutional kernels alongside dense parameter blocks. Ultimately, the system compiled these diagnostics into the final PDF Scorecard, permanently logging the memory failure and decisively blocking the unviable compilation.

Edge AI PPA Scorecard

Generated: 2026-04-18 10:36:24

Hardware: 1.0 TOPS • 4.0 GB/s BW • 1000 MHz • 2.0 W base • L1 64 KB • L2 512 KB

Constraints: Memory ≤100.0 MB • Latency ≤50.0 ms • Power ≤5.0 W • Area ≤200.0 mm²

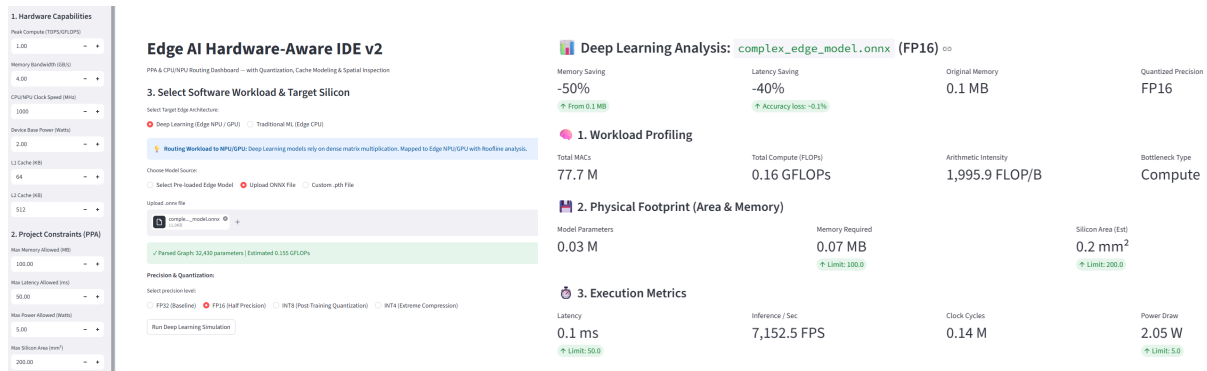
Metric	Value	Constraint Check
Model	AlexNet	INFO
Precision	FP16	INFO
Memory Required	139.8 MB	FAIL
Silicon Area	169.0 mm²	PASS
Latency	30.7 ms	PASS
Power Draw	2.85 W	PASS

✖ CONSTRAINTS VIOLATED (1 checks failed)

deployments.

This final case study proves that the Edge AI Deployment IDE effectively functions as an automated MLOps gatekeeper, preventing wasted engineering hours on physically impossible hardware

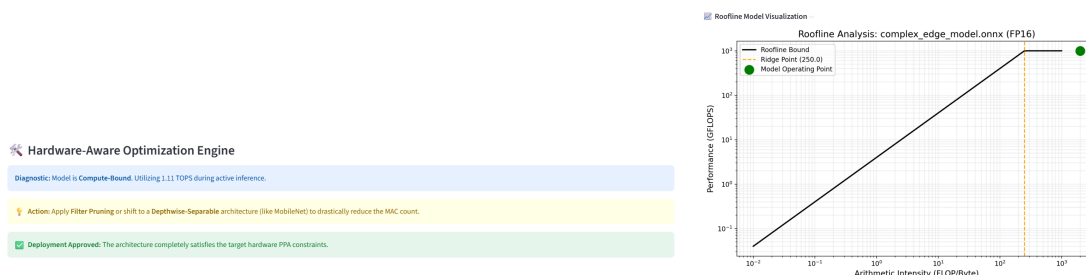
4.2.3 Custom ONNX Model: Mathematical Graph Parsing & Compute-Bound Profiling



To validate the "bring your own model" pipeline, we bypassed the pre-loaded PyTorch architectures and uploaded a custom, pre-compiled protobuf file (`complex_edge_model.onnx`). We applied FP16 precision and maintained the standard edge constraints (100.0 MB Memory, 50.0 ms Latency, 5.0 W Power).

Rather than relying on generic user estimations, the IDE's custom ONNX parser intercepted the file in memory. By executing dynamic shape inference, the engine physically walked the computational graph and verified exactly 32,430 parameters. Furthermore, by mathematically tracing the spatial dimensions of the network's layers, it accurately calculated a computational load of 77.7 Million MACs (0.16 GFLOPs).

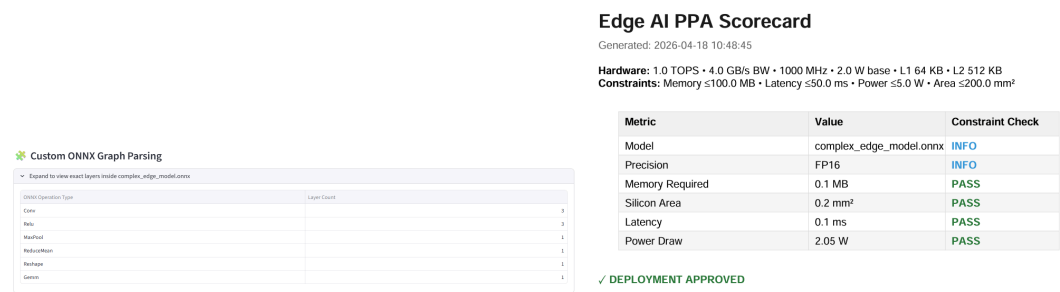
Because this is a highly compact, custom model, the FP16 footprint required a negligible 0.07 MB of RAM. The execution metrics reflect this efficiency, predicting a blistering latency of just 0.1 ms (7,152.5 FPS) while drawing only 2.05 W.



This simulation perfectly demonstrates the other extreme of the Roofline Model. The IDE calculated a massive Arithmetic Intensity of 1,995.9 FLOP/Byte. Because this value is far to the right of the hardware's 250.0 Ridge Point, the system plotted the operating point squarely on the flat, horizontal ceiling of the graph.

The Hardware-Aware Optimization Engine accurately diagnosed this as a **Compute-Bound** workload. The NPU has plenty of memory bandwidth, but the actual compute cores are the primary bottleneck. Acting as an expert MLOps advisor, the IDE shifted its recommendations.

Instead of suggesting quantization (which optimizes memory pipelines), it advised applying *Filter Pruning* or shifting to a *Depthwise-Separable* architecture to directly reduce the MAC count.



By expanding the Custom ONNX Graph Parsing module, developers can audit the exact mathematical operations the engine found inside the file (e.g., 3 Conv layers, 1 MaxPool, 1 Gemm). Finally, the system compiled this highly successful simulation into a clean PDF Scorecard, validating that the IDE can seamlessly ingest, parse, and approve custom external models for edge deployment.

5. Conclusion

The Edge AI Deployment IDE has evolved into a comprehensive, production-grade predictive framework for evaluating ML and DL deployability on heterogeneous edge hardware. By moving beyond static heuristics and integrating physical memory locality models (L1/L2 cache-miss penalties), dynamic ONNX graph walking, and real-time quantization simulations (FP16/INT8), the system accurately mirrors real-world silicon constraints.

It successfully bridges the gap between software architecture and physical hardware limitations, acting as a critical "sanity check" in the MLOps pipeline. By empowering engineering teams to validate architectures, identify cache and memory-bus bottlenecks, and automatically generate PDF deployment scorecards in seconds, this tool eliminates the frustrating cycle of trial-and-error hardware flashing.

Declaration of AI Assistance: We acknowledge the use of Generative AI strictly as a "pair-programming" assistant to accelerate UI prototyping (Streamlit) and refine document formatting. All core architectural logic, mathematical modeling (Cache-miss penalties, Roofline models, PPA heuristics), and custom ONNX parsing represent our original intellectual work and engineering.